

2)

Assembly

13.2.18

Stackta, program alusini
kisiye baska bir is yapacagini
görmekle ilgili yerleri.

Assembly programi:
 { stack } segmentleri
 { data }
 { code }

Yönelim	index	teslim (segment)	işaret (pointer)
ax	SI	CS	IP
bx	DI	DS	SP
cx		ES	BP
dx		SS	

Stack yapisi
LIFO var

Stackta gidip gelenlerin
nerede dandegini saklariz.

→ Kull, Data, Yipin den alur program.

→ CS yotmaci ⇒ Code segmentinin basladigi yeri gösterir.

→ DS yotmaci ⇒ Data " " " "

→ SS " " ⇒ Stack " " " "

→ ES " " ⇒ Extra segmenttir. Segment = Kesim.

⊗ Her kesim birk büyüklükte olabilir max. bit sayilarda hatirla

→ IP pointer ⇒ instruction (komut) CS:IP çiftligi

→ SP pointer ⇒ stack

→ BP pointer ⇒ base - Stackte caliriyor.

→ SI yotmaci ⇒ source index

→ DI yotmaci ⇒ Destination index

Data segment
ile
calisir.

SS:SP
SS:BP
DS:SI
DS:DI
DS:BX
Eğer ES kullanilacak
DS:SI
ES:DI
Çiftte kullanilir
Çiftte kullanilir
Çiftte kullanilir

Veri grubu

→ AX yotmaci ⇒ AH/AL = 2 tane 8bit veya AX = 16bit Accumulator.

→ BX " = Base register Temel aritmetik islemlerde kullanilir. Yeri gütelligiinde
BH/BL SI ve DI gibi kullanilabilir.

→ CX " = Count register aciklanir rotate, ötele sayma guclugunu islemlerde

→ DX " = Data register ⇒ DX/DH/DL bilhassa carpma ve bolme AX ile bir
veya felip 32 bit gibi olar. Bolme de her zaman tam sayi
bitleri olacak. I/O portlarinda numarası 256 dan büyük
olan portlar numaralandirilmadigindan DX kullanilir.

→ PSW ⇒ program status word (bayraklar) islen sonuclara dair ipuçlarını
berindir.

Fiziksel adres = Kesim adres x 16 + gözetli konum adresi

①

Assembly

20.2.18

Yazılıncıya göre yeteneceği → 16 bit reg. var 8086'da

Veri Grubu Registerleri

⇒ 16 bit veya 2x8 bit:

- AX → AH / AL → accumulator 4 bitinde kullanılır
- BX → BH / BL → Base register
→ Data adresi olarak
- CX → CH / CL → Count Register
→ Tekrarlı işlemlerde (loop) → CX yada CL tekrar sayısında
- DX → DH / DL → Data Reg.
→ Comma ve bölme işlemlerinde AX'ın yardımcıları
→ Carriage işlemlerinde DX AX'e soldan girer ve yığılma olarak
→ port adreslerinde 256'dan büyük adresler DX ile alınır.

Kesim Grubu Registerleri

⇒ Hepsi 16 bit. Parçası:

Data Segment → Üzerinde işlenilecek verilerin yığını

Code Segment → code ünitesinin yığını

Stack Segment → stack'inin yığını (yığılma) Son girilen ilk çıkar. ┌ - küç-küçler
└ + büy-küçler

Extra Segment → DS ile aynı amaçla kullanılır.

İsaretçi Grubu Yetenekleri (Pointer)

Instruction Pointer → çalıştırılacak olan komutun gösterici (Code Segment içinde). Değeri otomatik.

Stack Pointer → Stack alanına isaretçi. Değerini otomatik ayarlar.

Base Pointer → Stack için kullanılır. Değerini bizzat ayarlar. Stack içinde değeri okunak ve değiştirme hakkı var. Ama çıkarmaya yok.

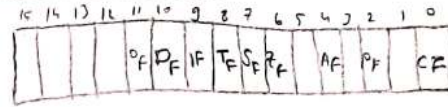
İndis Grubu Registerleri

Source Index → Veri kısmında veriye erişmek için kullanılır. ⊗ BX Indis
Register
kullanılır.

Destination Index → Aynı.

Program Status Word (Bayraklar / Flags) (16 bitlik word)

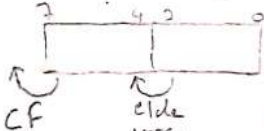
→ Yapılan işlemlerden sonra program durumunu gösterir.



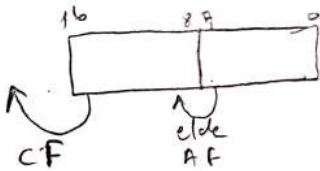
• CF = Carry Flag (Ekle)

Set = 1 / Clear = 0

• AF = Auxiliary Carry Flag



② 16 bit 12 7 den 8'e elden olursa AF set olur.



• IF = Interrupt Flag (Kese)

→ işlemciye bilgi yaptırarak oluyor

• OF = Overflow Flag

→ Arithmetik taşıma varını yokunu anlamamıza yardımcı olur

• PF = Parity Flag

→ Veri iletilirken hata olup olmadığını kontrol için kullanılır

→ 1 bitin sayısının tek/çift kontrolünde

• ZF = Zero Flag

→ Sonuç 0 ise, birşey birşeye eşit-i? sonuç 0 ise Zero Flag set (1) olarak

• SF = Sign Flag

→ Sonuç - ise set olur.

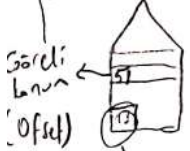
• TF = Trap Flag (Single Step Mode)

→ debugda kullanılır. Biraz yavaşlatır

• DF = Direction Flag

→ Yön belirler. İşlemin ileri/geriye mi yapılıp yapılmadığını gösterir. Kater kontrol biriminde kullanılır

→ (IR, SP, BP, SI, DI, BX)



Segment Adres (DS, CS, SS, ES)

$$\text{Fiziksel Adres} = \frac{\text{Kesin Adres} \times 16}{16 \text{ bit}} + \frac{\text{Görelî Konum}}{16 \text{ bit}}$$

(CS, IP)

(SS, SP), (SS, BP)

(DS, SI), (DS, DI), (DS, BX)

(DS, SI), (ES, DI)

Fiziksel Adres
Kesin adresi x 16
+ görelî konum

(3)

Assembly

20.2.18

Komutlar

Label : Mnemonic ^{Varış} ← ^{Kaynak} Operand1, Operand2 ; Yorumlar.

1-) Veri Aktarım KomutlarıOBJECT CODE

→ Hexadecimal

③ → Başrakların üzerinde etkisi Yok

örneğin mov op1, op2
 mov op1, op2

Sayı	Mnemonic	Object Code (Byte)	Clock Cycle
60	mov rb, daddr 8 bitlik hukukçi 1 register	2 byte + [disp][disp] byte tipi memory bölge	8 + EA → bellekten okuma Sayı, ismi - 1. deyimlerin paralel adresinin her bir için geçen süre.
61	mov rax, daddr 16 bitlik hukukçi 1 register	2 byte + [disp][disp]	8 + EA → bellekten okuma
61	mov daddr, rb	2 byte + [disp][disp]	2 + EA → belleğe yazma
61	mov daddr, rax	2 byte + [disp][disp]	2 + EA → belleğe yazma
61	mov sr, daddr	2 byte + [disp][disp]	8 + EA * sr = any of the segment registers.
61	mov daddr, sr	2 byte + [disp][disp]	2 + EA
68	mov daddr, data8	3 byte + [disp][disp]	10 + EA * data8 = 8 bitlik ifade editör verit. 0-255 örneğin sayı = 10
68	mov daddr, data16	4 byte + [disp][disp]	10 + EA * data16 = 16 bitlik ifade örneğin sayı = 65530
68	mov rb, data8	2 byte	4 * mov al, 1
68	mov rax, data16	3 byte	4 * mov ax, 1
75	mov rdi, rbs	2	2 * rdi = 1 bytelik register destination rbs = " " source mov al, bl
	mov rdi, rax	2	2 * mov ax, bx
	mov sr, rax	2	2 * mov ds, ax
	mov rax, sr	2	2

* Instruction
kullanılmıyor
Dikkat

* Dikkat	mov data8, rb	mov rax, rb	mov rdi, rax
	mov data16, rax	<u>Yok</u> 16 bitlik bisi	<u>Yok</u>
	byte bisi <u>Yok</u> - 5 peler	uydur	

NOT operand 1, 2, 3
uydur - 1. gördükte

* operand 1, 2, 3 uydu - 1. gördükte

(4)

assembly

20.2.18

XCHG Komutu $\Rightarrow$ xchg op1, op2

Syfa	Mnemonic	Object Code	Clock
61	xchg rb, dword	$2b + [disp][disp]$	17+EA
	xchg rw, dword	$2b + [disp][disp]$	17+EA
75	xchg ax, rw	1b	3
	xchg rb, rb	2b	4
	xchg rw, rw	2b	4

op1	op2
3	5
5	3

al=3 bl=5
 mov al, al
 mov al, bl
 mov bl, cl

byf	clot
2	2
2	2
2	2
6	6

(75) xchg ax, bx $\rightarrow$ 2b/3c
 xchg bx, ax $\rightarrow$ 2b/4c

xchg ax, bx
 2by 4clot

Dikkat! Optimize gel.

$$\frac{OR}{5} = \frac{al}{5} \quad \frac{sayi}{3}$$

mov temp, al
 mov al, sayi
 mov sayi, temp

mov dword, dword $\Rightarrow$ Yok böyle bir şey.
 Hoca kıracak puan. $\nabla \nabla \nabla$

mov sayi, [esi]
 mov [esi], [edi] ∇ Hata

LEA Komutu (Load Effective adress)

$\rightarrow$ İşlemciye doğru alıyorsun Load denildiğinde ilave olarak store da memorye doğru.

-	7	0
13C7h	75h	← SayıB (byte tipi)
13C8h	45h	← SayıW (word tipi)
13C9h	65h	
+		

Lea si, sayib $\Rightarrow$ sayib'nin adresini si'ye yazıyor.

$\Rightarrow$ // si $\leftarrow$ 13C7h
 $\hookrightarrow$ tipler farklı olsa pekiştirilmesin. adresler.

lea di, sayiw $\Rightarrow$ // di $\leftarrow$ 13C8h

lea si, [13C8h] $\Rightarrow$ // mümkün.

sayiw = $\begin{matrix} \text{high} \\ \text{low} \end{matrix}$ 65h, 45h

$\rightarrow$ data segment $\rightarrow$ extra say.
 LDS komutu LES komutu

Belliğin 1 adresinden itibaren segment ve pointer konumunu atamamın sağlıyor.

LDS si, sayiw
 ds $\leftarrow$ [EA sayiw+2] $\leftarrow$ [13C8h+2h] $\leftarrow$ [13CAh]
 si $\leftarrow$ [EA sayiw] $\leftarrow$ [13C8h]
 si $\leftarrow$ 6545h ds $\leftarrow$ 3555h

LES di, sayib
 di $\leftarrow$ [EA sayib] $\leftarrow$ [13C7h] $\leftarrow$ 4575h
 es $\leftarrow$ [EA sayib+2] [13C9h] $\leftarrow$ 5565h

27.2.18

LEA SI/DI/BX 16.57 reglar operandi altele.

700
 115 64 32 16 8 4 2 1

1	1	0	0	1	0	0	0
---	---	---	---	---	---	---	---

 +
 700

1	1	0	0	1	0	0	0
---	---	---	---	---	---	---	---

 115 64 32 16 8 4 2 1
 1 1 0 0 1 0 0 0

⑩ optisch tieferer ungenutzter Bereich
2 side bytes 2 side byte
1. wurde 2 side

(1) add (al) data = 8
2 byte tr, 4 ch. lg.
(2) add (rb) data = 8
3 byte , 4 ch. lg.
NOT = Burde of file
disgr rb waste
of space by ah

sf			
62	add rb, doaddr	2 + [0x15F][EIP]	3 + EA
62	add rx, doaddr	"	3 + EA
62	add doaddr, rb	"	16 + EA
62	add doaddr, rx	"	16 + EA
70	add al, data8	2	4
	add ax, data16	3	4

$\Rightarrow$ inc al
inc rb

2 byte/3CCy. $\equiv$ add r1, d1, r8
3B/4C

2

Assembly

27.2.18

- NEG op1 // işlem sonucu $op1 = -op1$ // tek operandlı Dikkat
// 2'ye tene işlemidir // Bu komut örneği:
- CMP op1, op2 // $(op1 - op2)$ sonucu bir yere atılmıyor!
// sonuç bayraklarda. @ BAYRAKLARDA !!!
- MUL OP_{8bit} // $AX = AX * OP_8$ // Tek OPERAND Dikkat
- MUL OP_{16bit} // $DX:AX = AX * OP_{16}$ // mul 15 → Bu komut r15'te
// 16 bit 15'in tipi belli değil.
// mul carpma parçatılacağı carpma için.
// byte'ları kullanır belli değil.
- IMUL OP₈ // mantığı mul la aynı OP 8 bit se al ile; 16 bit se
IMUL OP₁₆ // ax ile carpma yaptırır isaretili sayı carpma olur!!!

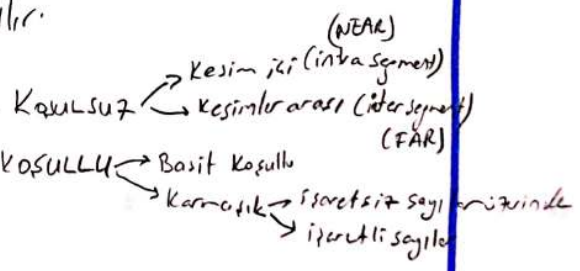
→ DIV OP₈ // $\frac{AX}{OP_8} \Rightarrow$ AL'de bölün AH'da kalan yer alır!!!
 $0A = mov ax, 5$
 $mov bx, 2$
 $div bx // ax/bx \Rightarrow al=2 \quad ah=1$

→ DIV OP₁₆ // $\frac{DX:AX}{OP_{16}} \Rightarrow$ AX'te bölün DX'te kalan
 // Div komutu + isaretili sayılarda kullanılır.

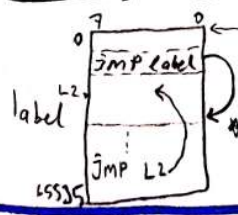
→ IDIV OP // isaretili sayılarda kullanılır.

DALLANMA KOMUTLARI

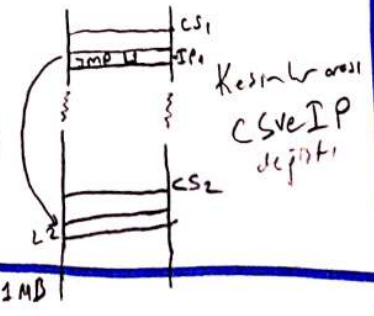
Basit koşullu → tek bir bayrağın durumuna bakar
 Karmaşık → 1 tane bayrağın durumu veya daha
 mantıksal ilişki...



Kosulsuz → JMP salın (u) yok



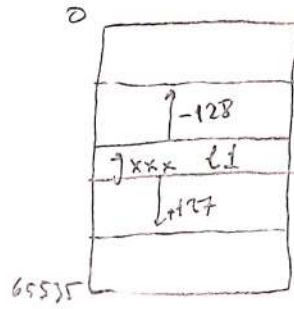
JMP → IP'indeğini değiştirir.
 Kesim içi



3

Kasullular

J_{xxxx} label $\rightarrow$ far değildir.
Atlana aralı 1 byte'tir.



$\Rightarrow$ IP değeri ya
128 a kadar ya da
127 artar. Bu
husus örnek: ///...

Basit Kasullular

JE/JZ (Equal/Zero) $\rightarrow$ Zero flag 1 ise sonuç 1 olur ise
kasul sağlanır oluyor.

(not equal/not zero) JNE/JNZ $\rightarrow$ Zero flag 0 ise sonuç 1 olur ise
kasul sağlanır oluyor.

JS (on sign) $\rightarrow$ Sign flag 1 ise sonuç 1 olur ise
kasul sağlanır.

JNS (not sign) $\rightarrow$ SF 0 ise

JO $\rightarrow$ Overflow flag 1 ise kas. sağ.

JNO $\rightarrow$ " " 0 " " "

JP/JPE $\rightarrow$ Parity flag 1 ise (even parity)

JNP/JPO $\rightarrow$ parity F. 0 ise (odd parity)

JC $\rightarrow$ Carry 1 ise

JNC $\rightarrow$ Carry 0 ise.

İşlemler
sırasında $\left[\begin{array}{l} \text{AL} \text{ TINDA} / \text{LISTONDE} \\ \text{BELOW} / \text{ABOVE} \end{array} \right.$

İşlemler
sırasında $\left[\begin{array}{l} \text{BÜYÜK} / \text{KÜÇÜK} \\ \text{GREATER} / \text{LESS} \end{array} \right.$

$JB/JNAE/JC$ $\rightarrow$ CF=1 ise kasul gerçekleşir.

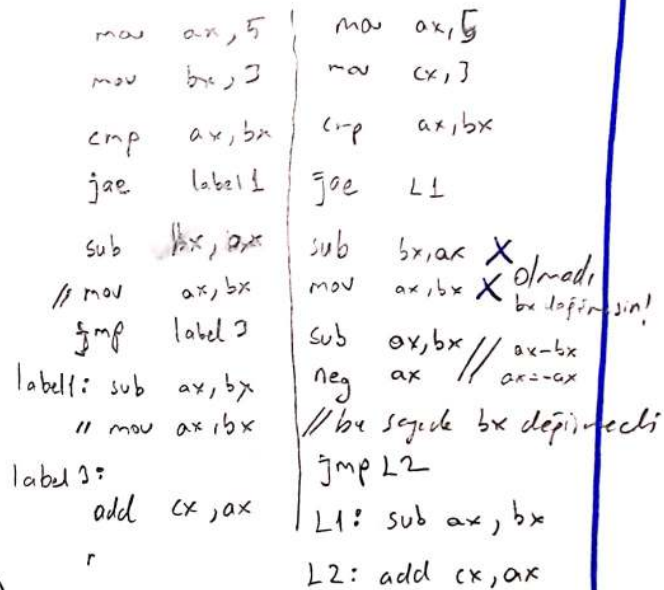
$JA/JNBE$ $\rightarrow$ CF=0 $\hat{=}$ ZF=0 ise kasul sağlanır.

$JAE/JNB/JNC$ $\rightarrow$ CF=0 ise kasul sağlanır.

27.2.18

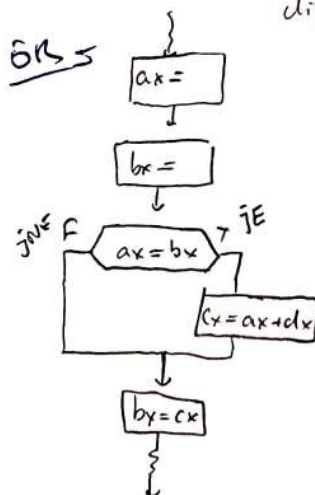
$$j_L / j_{NGE} \rightarrow \text{sign} F \neq 0 \text{ of } \left[\hat{v}_e \right] \text{ } \mathcal{F} = \phi$$
$$j_{NL}/j_{GE} \rightarrow SF = OF$$
$$jLE/jNG \rightarrow SF \neq OF \wedge ZF=1$$
$$j_6/j_{NLE} \rightarrow ZF=0 \wedge SF=0F$$

(A) Bu konutlar
Campocin.
desen, oda bittir.



L1: // Sakin DIII

registri kullanak
yasaq depil. a-a
oreg. da baska
deyer var mı yokmu
dite bakenaligiz!!!

$$\begin{array}{l} -(ax-bx) // \text{ simplifizieren} \\ -ax + bx \end{array}$$


```

mov     ax,
mov     bx,

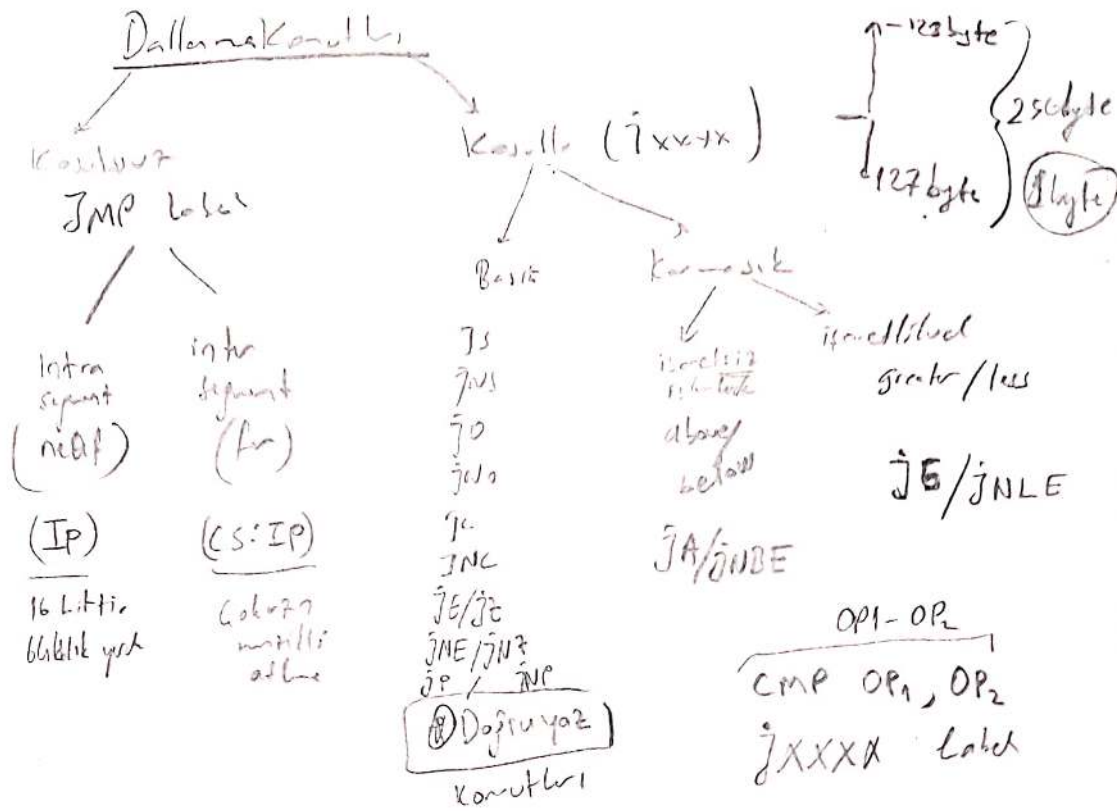
cmp     ax, bx
jne     L1
mov     cx, ax
add     cx, dx
L1: mov     bx, cx

```

①

Assembler

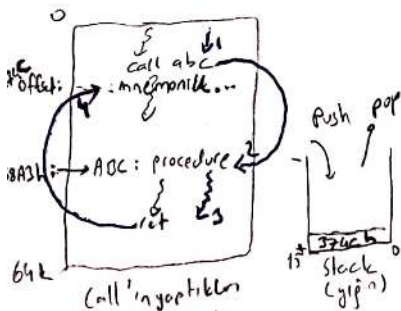
6.3.10



YORDAM GAGIRMA

- Nasıl döreceğine dikkat et!
- CALL komutu ile yordam çağırılır. $OP = (CALL \text{ yordam-adı}) / RET$
- RET / RETF ile dönüş yapılır.

Yordam çağırma



- Call'ın geri dönüş noktası
- 1) Dönüş için pointer (offset) konumu yapın (push)
- 2) IP'ye offset (abc)
- returnun yapılması
- 1) Dönüş adresini yapın at IP'ye long
- POP IP

- Kesimci intrasegment (Near)
- yığın kullanılarak
- yığından 1 sefer
- 16 bit kayılır/almaz
- SP değeri stacke girince 2 byte artar.
- stackten almaya yarar.

push ve pop call ve ret kullandığı

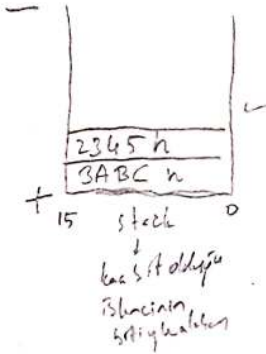
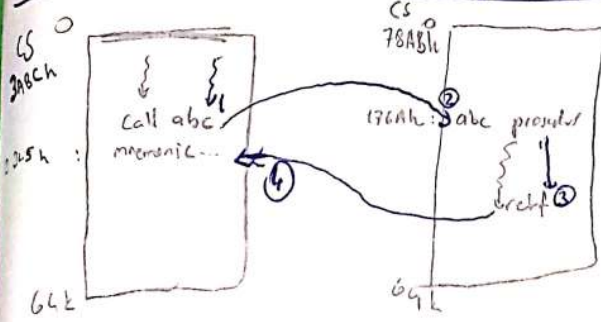
(B)

Assembly

6.3.12

kesimle arası Yordan çağırma

(2 farklı code segment arası)



→ farklı kesime girtili!!

→ Callın yapıldıkları

1-) Dönüş adresinin değeri yığına koy.

(CS değeri ve kodunun başlatıldığı segment ve offset değeri)

2-) CS ← segment (ABC) (78ABh)

IP ← offset (ABC) (176Ah)

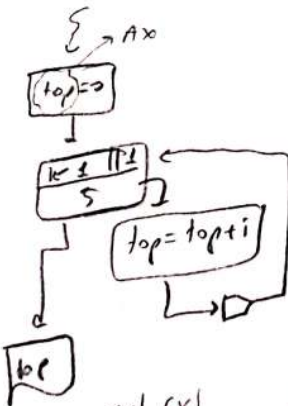
→ retf'in yapıldıkları1-) önce pop IP (offset) IP ← 2345h
Sonra pop CS (segment) CS ← 78ABh

GEVRİMLER

CX = tutarlı bloklarda sayı tutmayı sağlıyordu.

Geçmiş LOOP label ile yapılır

→ loop arka planda CX registerinde değeri 1 azaltır. CX ≠ 0 iken döngü devam eder.



Burada hata yok, bir analiz hatası

```

mov ax, 0
mov cx, 5
cli: add ax, cx
loop cli

```

olması için kal

```

mov ax, 0
mov cx, 5
mov si, 1
cli: add ax, si
inc si
loop cli

```

ax	CX	top	i
0	5	0	1
5	4	1	2
0	3	3	3
12	2	6	4
14	1	10	5
15	0	15	

sonuç aynı ama ara değer farklı

① indisi olarak SI, DI, BX kullanırsak iyidir.

6.3.18

$$\begin{array}{r} \underline{Cx} \\ 5 \\ 3 \\ 2 \\ 1 \\ 0 \\ -1 \\ -2 \\ -3 \\ -4 \\ -5 \end{array}$$

$\begin{array}{r} \text{C x} \\ \hline 5 \\ 3 \\ 2 \\ 1 \\ 0 \\ 5 \\ 1 \\ 3 \\ 1 \\ 0 \\ 4 \\ 7 \end{array}$



```

    [A]
    mov si, 1
    mov cx, 15
L2:  [B]
      push cx
      mov di, 1
      mov cx, 3
L1:  [C]
      inc di
      loop L1
      pop cx
      [D]
      inc si
      loop L2
      [E]

```

→ bei ex. fkt. der 2, 3, ... ausl. nach
mit geschl. tipende loop bis zu jedem el. z
Ex in Kontrolle bis zu

Bayraklarla ilgili Konular

Set 1
Clear ϕ

op
read
sit

CLC | $CF \leftarrow 0$
STC | $CF \leftarrow 1$
com
RMC | $CF \leftarrow \overline{CF}$

$CL_i \mid IF = \emptyset$ $CLD \mid DFe \neq \emptyset$
 $St_i \mid iF = 1$ $STD \mid DFe = 1$

Auxiliary sign overfly parity } Bunbra Yole

oper and src { SAHF (Store) flag = ah
LAHF (Load) flag = ah

pushf 16 bitlik RPSW
popf 4 byte stack pointer

AH

7	6	5	4	3	2	1	0
SF	ZF	?	AF	?	PF	?	CF
1	0	0	0	0	1	0	0
8h				4h			

```
mov ah, 86h
saf
```

4

Assembly

6.3.18

Mantıksal Komutlar

- AND OP1, OP2 OP1 ← OP1 AND OP2
- OR OP1, OP2 OP1 ← OP1 OR OP2
- XOR OP1, OP2 OP1 ← OP1 XOR OP2
- NOT OP1
- TEST OP1, OP2 → operatör değişim & sonuç
bayraklarda olur.

↳ (OP1 AND OP2)

TEST AH, 8h

mov ax, 0 ≡ XOR ax, ax

mov ax, data6
Rw XOR rwi, rw
XOR RWD, RWS

3B/4C

2B/3C

Bayrak değişir.

Bayrak değişir.

①

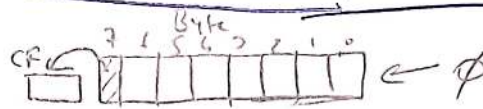
Assembly

13.3.18

Shift (Shift) Döndürme (Rotate) Komutları

SHL (Shift Left)

SHR (Shift Right)



⊗ SHL altında

Teknik olarak
2 ile çarpma
denir.

⊗ SHL AL, 1
AL yi 1 ötele.

⊗ shl ile çoklu
işlem yapılırken ⇒

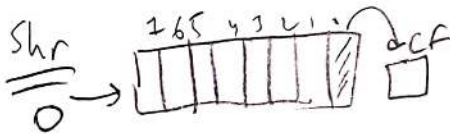
mov cl, 2
shl al, cl

ör = 3 = 00001000 B
11111111
00001000
0 "16"

⊗ shl al, 2
geersitdir

CL 255'e kadar
olabilir 0'dan byte ise
Seyet 0'dan word ise
cl 2-15 arası olabilir

mov cl, 10
shl ax, cl } → ax'i
10 kere
sola ötele.



⇒ shr 2 ye bölme dir. cf'de
kalır dur.

ör = 16 bitlik bir registerda kaç adet 1 vardır binary?
oluk 0?

xor bl, bl
mov ax, 3CFAh

L1: shr ax, 1
jnc L2
inc bl
L2: loop L1

0011 1100 1111 1010

NOT = Bu iş
test ile de
yapılabilir
Dene

xor bl, bl
mov cx, 16

L1: shr ax, 1
adc bl, 0
loop L1

xor bl, bl

mov ax, 3CFAh

mov cx, 16

mov dx, 1

L1: test ax, dx
jz devam
inc bl
devam: shl dx, 1
loop L1

Adc op1, op2 ⇒ op1 = op1 + op2 + cf

②

Assembly

13.3.18

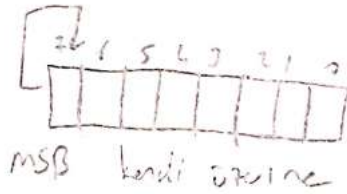
İşaretili sayılarda $\Rightarrow$ SAR, SAL

sağdan
right

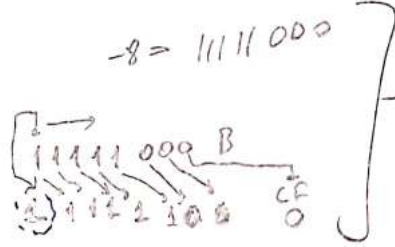
sağdan
left

SAL = SHL ile
aynı çalışır.

SAR $\rightarrow$



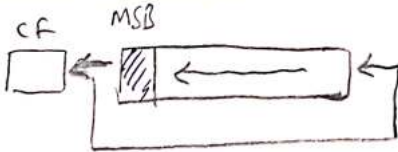
dönüşüm
bölme
2'ye



Sayı soldan
1 atılmış
pişir.

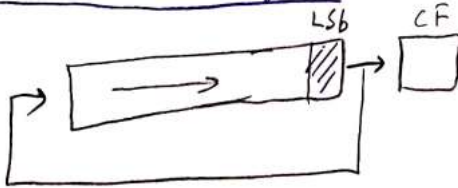
Dönüşüm Komutları

ROL (rotate left)



⊕ msb hem cf'ye girer hem de
lsb olur.

ROR (rotate right)

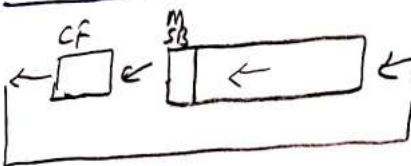


⊕ lsb hem cf'ye girer hem de
msb olur.

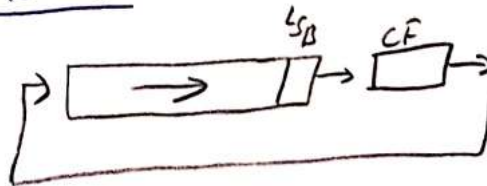
önceki sf'de toplama

shl ax, 1 yerine ror/rol ax, 1
yazılabilir.

RCL ⊕



RCR

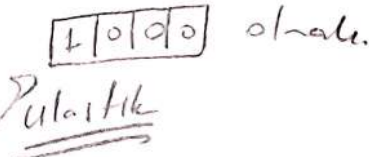
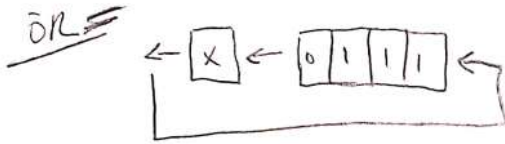


⊗ RCR ve RCL
çalışmazlar.

(3)

Assembly

13.3.18



- 1) 0 111 X
- cmc 1
- 2) 1 11 X 1
- cmc 0
- 3) 1 1 X 1 0
- cmc 0
- 4) 1 X 1 0 0
- cmc 0
- 5) X 1 0 0 0

NOT
5. deyim
yaptık
bit segment

```

mov ax, 2875h
mov cx, 17 ; (16 bit + 1)
cli: rcl ax, 1
jc clear
stc
jmp devam
clear: cbc
devam: loop cli

```

Karakter (String) Komutları

⊛ Bu komutlarda direction flag önemlidir. DF=1 ⇒ down
DF=0 ⇒ up

```

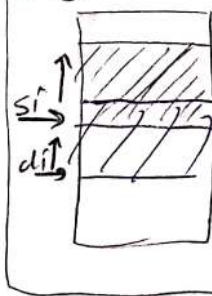
mov ax, 2785h
mov cx, 17
cli: rcl ax, 1
cmc
loop cli

```

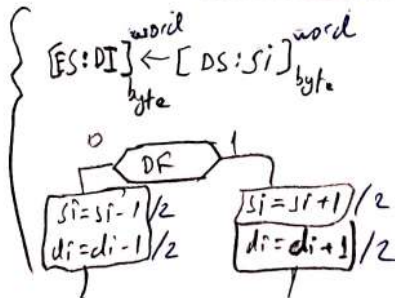
movs {movsb, movsw}

cmps
scas
lods
stos

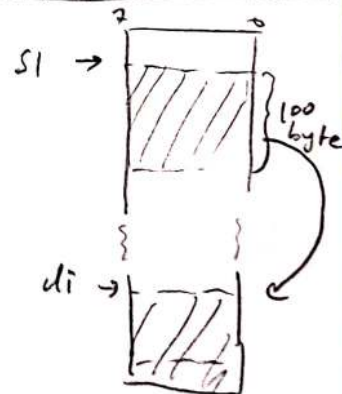
2. Sıraya



SI
DI
CX
movsb



1. Sıraya



```

mov cx, 100
lea si, di+1
lea di, di+2
mov [di], [si]
cli: mov al, [si]
mov [di], al
inc si
inc di
loop cli

```

NOT = String komutları operand almaz lar!

(4) es:di, ds:si

assembly

13.3.18

CMSB

$\left\{ \begin{array}{l} [DS:SI] - [ES:DI]_{\text{byte}} \\ \text{DF?} \end{array} \right.$ $SI = SI \mp 1$
 $DI = DI \mp 1$

NOT =

Bundocun sonra

kasulla dallerma

kontrollu gelmesi

beklenir.

CMSW

$\left\{ \begin{array}{l} [DS:SI] - [ES:DI]_{\text{word}} \\ \text{DF?} \end{array} \right.$ $SI = SI \mp 2$
 $DI = DI \mp 2$

SCASB

$\left\{ \begin{array}{l} AL - [ES:DI]_{\text{byte}} \\ \text{DF?} \end{array} \right.$ $DI = DI \mp 1$

⊕ sonra sayraklarda olusur.

operandlarda defisitelek olmaz

SCASW

$\left\{ \begin{array}{l} AX - [ES:DI]_{\text{word}} \\ \text{DF?} \end{array} \right.$ $DI = DI \mp 2$

⊕ compare gitsidir aslinda bu ko-
mutlardan sonra kasulla dallerma
kontrollu beklenir

LDSB

$\left\{ \begin{array}{l} AL \leftarrow [DS:SI]_{\text{byte}} \\ \text{DF?} \end{array} \right.$ $SI = SI \mp 1$

⊕ Load iletmeceye
dogru alma

⊕ Store memory e
dogru yollama

LDSW

$\left\{ \begin{array}{l} AX \leftarrow [DS:SI]_{\text{word}} \\ \text{DF?} \end{array} \right.$ $SI = SI \mp 2$

STOSB

$\left\{ \begin{array}{l} [ES:DI] \leftarrow AL_{\text{byte}} \\ \text{DF?} \end{array} \right.$ $DI = DI \mp 1$

REP → Bu tek satirlik
looplari bi coktur

STOSW

$\left\{ \begin{array}{l} [ES:DI] \leftarrow AX_{\text{word}} \\ \text{DF?} \end{array} \right.$ $DI = DI \mp 2$

AL

$\left. \begin{array}{l} \text{mov cx, 100} \\ \text{rep stosb} \end{array} \right\} = \text{LI: movsb} \\ \text{loop LI}$

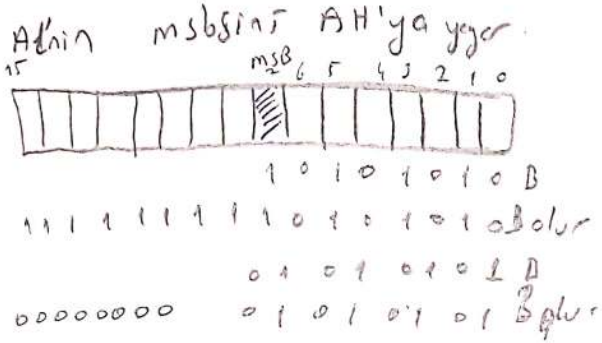
5

assembly

13.3.18

-CBW (convert byte to word)

$AX \leftarrow AL$



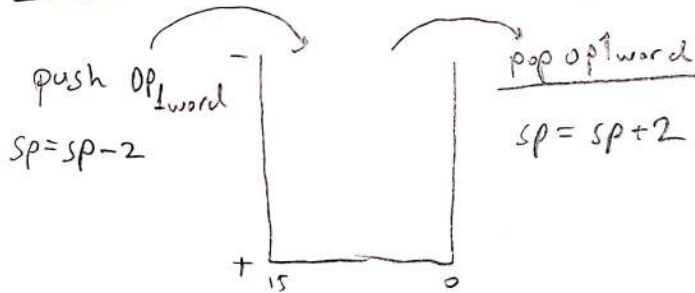
CWD (convert word to double)

Aynı CBW gibi.

İkinci de operand alınır

⊗ isaretli ise isareti muhafaza eder
çalışır

YİĞİN KOMUTLARI

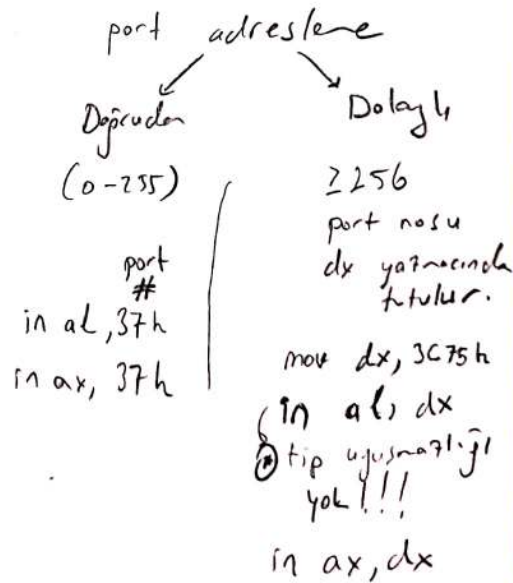


⊗ pushf ve popf
psw yı stacke
koyar/alır.
operand almaz.

GİRİŞ / ÇIKIŞ KOMUTLARI

65536 port var

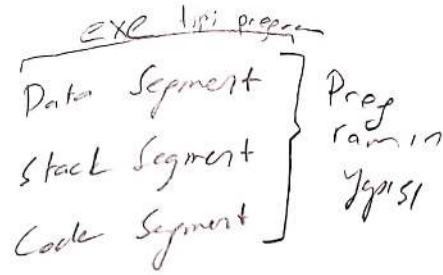
In / out



(1)

Assembly

27.3.18

DEBUG
ortamını
öğren!!2 tip assembly → exe
→ comExe tips→ Bu üç segment derinde
(bellekte)

Bellekte nasıl yapılır.

```

mystack SEGMENT PARA STACK 'yığın'
        DW 20 DUP(?)

```

```

mystack ENDS

```

```

mydata SEGMENT PARA 'veri'
        sayı1 DB ?
        toplam DW 75

```

```

mydata ENDS

```

```

mycode SEGMENT PARA 'kod'
        ASSUME SS:mystack, DS:mydata, CS:mycode

```

```

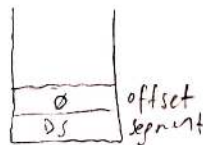
ANA PROC FAR // assembly programı far ile belirtilir.

```

```

Değerler:
{
    PUSH DS
    XOR AX, AX
    PUSH AX
    MOV AX, mydata
    MOV DS, AX
    ...
}
mnemonicler } stack'e pushlanan
               poplanabilir.

```



```

        RETF
ANA ENDP
; // farklı prosedürler

```

```

mycode ENDS

```

```

END ANA

```


2

Assembly

27.3.18

Örnek 1 =

2 tane byte sayisi
toplayin → 27.3.189

// Agri sayi wordlik alinir
da yop → 2703189

```
ms    SEGMENT PARA STACK 'stack'  
      DW 2D DUP(?)
```

```
ms    ENDS
```

```
MD    SEGMENT PARA 'data'
```

```
      sayi1 DB 0FDh
```

```
      sayi2 DB 0AFh
```

```
      toplam DW 0
```

```
MD    ENDS
```

```
MC    SEGMENT PARA 'code'  
      ASSUME DS:MD, CS:MC, SS:MS
```

```
BASLA PROC FAR
```

```
      PUSH DS
```

```
      XOR AX, AX
```

```
      PUSH AX
```

```
      MOV AX, MD
```

```
      MOV DS, AX
```

```
      XOR AX, AX
```

```
      MOV AL, sayi1
```

```
      MOV BL, sayi2
```

```
      ADD AL, BL
```

```
      ADC AH, 0
```

```
      MOV toplam, AH
```

```
      RETF
```

```
BASLA ENDP
```

```
MC    ENDS
```

```
END BASLA
```

3

Assembly

27.3.18

* Kendim 64 bitlik

2 sayıyı carper assemblyi yazmalıyım

* Cyi assemblyi ye karıngı arastır

Örnek 3

Verilen dizideki çift ve tek değer sayısını bulalım.

myS segment para stack 'yığın'

dw 20 dup(?)

myS ends

myD segment para 'data'

dizi db 1, 2, 7, 8, 13, 64, 62, 19, 39, 66

eleman db 10

tek db 0

çift db 0

myD ends

myC segment para 'kod'

assume ds:myD, ss:myS, cs:myC

tekçift proc far

push ds

xor ax, ax

push ax

mov ax, myD

mov ds, ax

xor cx, cx

mov cl, eleman

xor di, di

→ dngui: mov al, dizi[di]

shr al, 1

adc tek, 0

inc di

loop dngui

mov al, eleman

sub al, tek

mov çift, al

retf

tekçift endp
myC ends

end tekçift.

→ Bu sorunun tek sayı toplama ve sayıların kareleri part.

①

Assembly

4.4.18

my-ss segment ... para stack 'ypin'

DW 15 DUP(?)

my-ss ends

my-ds segment para 'data'

dispatch to dup(?)

denon due to

my-ds ends

my-ss segment para 'code'

assume ss:my-ss, ds:my-ds, cs:my-cs

one proc for ... the main ypin. we total for oluchyd

// don't address ypin here because

push ds
xor ax, ax
push ax
// don't address ypin here because
// we execute program ypin here.

mov ax, my-ds
mov ds, ax
// don't use here because we use it in



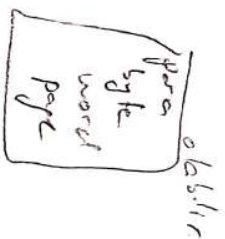
retf

ana endp

my-cs ends

end ana // main part oluchyd code ypin here.

// initial 1 data for first case. take to write
// in sample end ana dypc write



2

Assembly

2.6.18

OR = 10 adet word elemanıdır. bu elemanların
değerlerini. yani stringlerini al.

myss segment para stack 'yigin'

dw 15 dup(?)

myss ends.

myds segment para 'data'

dişi1 dw 5, 7, 9, 11, 13, 15, 17, 19, 21, 23

dişi2 dw 10 dup(?) // bu köşe herden sonra köşeler K olacak yoranda.
elen dw 10 // her 2 dup. 105 boy.

myds ends.

myc segment para 'code'

assume cs:mycs, ds:myds, ss:myss

ana proc far

move cx, elen

for si, si

move di, elen → move di, elen
shl di, 1
sub di, 2

1: mov cx, dişi1[si]

mov dişi2[di], cx

sub di, 2

add si, 2

loop 11

↑ köşe olan 1 - 5
// dişi1[si], dişi2[di] dişi-5 getirir
sol kod doğru.

köşe olan köşe 2.

dişi dw 10 dup(?)

elen dw 10

move cx, elen

for si, si

11: push dişi[si]

add si, 2

loop 11

move cx, elen

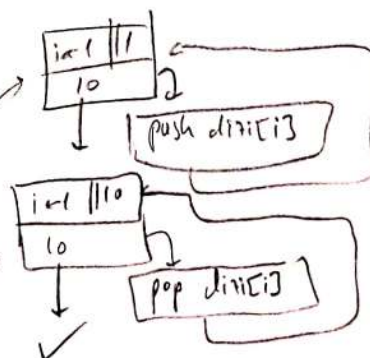
for si, si

12: pop dişi[si]

add si, 2

loop 12

// yığın enuz dişi + 10 word kadar büyüklüktedir.



(3)

Assembly

4-6-18

gizel cözen

03a18a-asm

xor si, si

mov di, elem

shl di, 1

sub di, 2

mov cx, elem

shr cx, 1 // cx = cx/2

!!: mov ax, di[si]

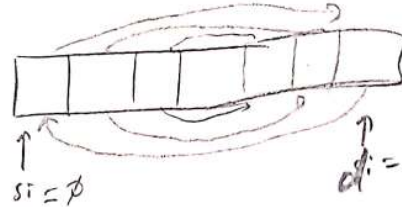
xchg ax, di[si]

xchg ax, di[di] // move di[di], ax

add si, 2

sub di, 2

loop !!



di = elem - sayi * 2 - 2

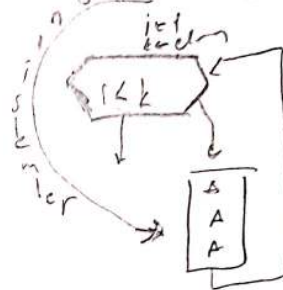
k = elem

sayi	1	1	1
elem/2			

```

temp = di[si]
di[si] = di[k]
di[k] = temp
k = k - 1

```



V:2

xor si, si

mov di, elem

shl di, 1

sub di, 2

!!: cmp si, di

jge son // si >= di ise bitir.

mov ax, di[di]

xchg ax, di[si]

mov di[di], ax

add si, 2

sub di, 2

jmp !!

son:

K

xor si, si

mov ax, di[si]

ax = 0001

Ami iflu

lea si, di

mov ax, [si]

ax = 0001



(1)

Assembly

4.4.18

COM TIPI ASSEMBLY PROGRAM

- exe dosya olarak çalıştırılabilir

→ TEK SEGMENT var. hepsi çalışık diğilirdi

(CS, DS, SS aynı blokta)

CS:DS:SS



⇒ PSP de girilen adresler, silenti gittikçe değişecek sayılar falan var. burayı kullanırız.

⇒ Stack baştan itibaren 0000 deyi bulunduru. Bu yeni değişim adresidir.

⇒ Stackteki diğer adresi 1 adet olduğunda diğer adresler near prosedür tanımlanıyor. Biri ekledik. Biri kaydediyor. Biri kaydediyor.

⊙ PSP yapabileceği bölge. burayı kullanırız.

⇒ data-code bunları 2 side başta yazabiliriz. Tercihen böyle.

myseg segment para 'code'

org 100 h // ilk 256 byte'ı okunabilir sağladı. offset 256 yaptık.

assume ss:myseg, cs:myseg, ds:myseg

baba proc near

ret

// ds falan initialize etmeye gerek yok. exe'deki ilk 5 satır okunabilir de olur. Fakat yazmaya gerek yok.

baba endp

{Veri tanımları}

myseg ends

end baba // C'deki main gibi burada main olduğunu (babanın) anlattık.

5

assembly

u.l. b

→ burada data segmentinin başta olduğu con tipi program yazacağız. Bir öncekinde de seneler yapıldı.

myseg segment para 'code'

org 100h

assume cs:myseg, ds:myseg, ss:myseg

Veri: jmp baba

} veri tanımları

baba proc near

...

ret

baba endp

myseg ends

end Veri

ÖN → word tipinde 1 dizide belli 1 eleman kaç tane olduğu bulunur.
dizi dw aranan dw sayac (dizi boyutu?)

myseg segment para 'code'

org 100h

assume cs:myseg, ss:myseg, ds:myseg

but proc near

mov cx, eleman // dizinin eleman sayısı

xor si, si // başta ilk dizinin 0

mov ax, aranan // aranacak eleman dizinin ortasına

11 cmp [si], ax // dizinin elemanı arananı karşılaştır.

jne devam

inc sayac // her itir sayacı artır

devam: add si, 2 // dizinin word arttırma cift

loop 11

ret

but endp

devam dw ...

12i dw ...

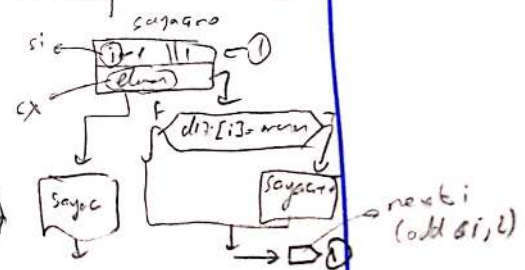
myseg ends

end but

değiş meşşş exe con kipsi dahil

ÖZET
030418b.asm

2 tane 030418b.asm



buun
exe knt
030418d.asm

Exe con dan
data fazla yer
tutar. her zaman
exe de fazla
code alanı kullanılır.

dağılır

(1)

Assembly Self Study 5.4.18

Yatma Döğel Adresleme

mov si, offset mydata // si ← 0002H

mov ax, [si] // ax ← 4A13h

* Veyga

	0004H
4AH	0003H
13H	0002H ← mydata
	0001H
	0000H

lea di, mydata // di ← 0002h

mov ax, [di] // ax ← 4A13h

Base Relative Adresleme

→ operandin bellekteki adresi bx yardımıyla.
 → BX e farklı değere göre ilave edilerek bellekteki farklı adreslere erişilebilir min-birlik.

→ Bellekteki diziyi Bk birimle kullanılır.

ör = Mesajı DB 'Assembly'

Lea bx, mesajı // bx ← 0010h

mov al, [bx+4] // ax ← 4Dh // ax ← 'M'

0017h	59h	'I'
0016h	4Ch	'L'
0015h	42h	'B'
0014h	4Dh	'M'
0013h	45h	'E'
0012h	57h	'S'
0011h	53h	'S'
0010h	41h	'A'

mesajı → 0010h

Döğrudan İndirli Adresleme

ör = mydata db 41h, 53h, 53h, 45h, 4Dh, 42h, 4Ch, 59h

⊙ Aynı C dilinde olduğu gibi. mydata dizinin başlangıcıyla başlanır.
 dizi başlangıcı '0' indeksi ile olur.

⊙ Bu diziye erişmek için kullanılacak registerler: si, bx

xor si, si // si ← 0000h

mov al, mydata[si] // al ← mydata[0] // al ← 41h // al ← [EA + SI]

ör = burak dw 7893h, 1227h, 6060h, 3060h.

xor si, si

mov ax, burak[si] // ax ← 7893h // 4 byte dikkat et // aslında 4 byte, local yazar.

107h	59h
106h	4Ch
105h	42h
104h	4Dh
103h	45h
102h	57h
101h	53h
100h	41h

← mydata

Farklı, lea si, burak // efektif adresi sağlar.
 mov ax, [si]

lea si, burak
 mov ax, word ptr [si] // burak adresine.

(2)

Assembly Self Study 5.4.18

SÖZDE KOMUTLAR

- program diktlenmesine yareliltilir.
- bayraklar içinde ellisi yok
- debyta calistirilene
- veri terimlerinde, kesin diktlenmede, 1st uzerinde dosyanin diktlenmede kullunilir.
- genel amali ve kullunulan kullunulan olanlar 2'ye ayrilir.

Mnemonic komutlar

- makine kodu karsiligi var
- bayraklar da ellisi yapar
- debyta calir.

Macro Zamban

Yordam terimleri

Veri terimleri

Page (opsiyonel)

1st uzerinde dosyanin
satin-sutuna belirlenir
ör = Page 60, 180

Genel amali Sözde komutlar

① Dosya terimleri kullunulanlar

Title: 1st uzerinde
dosyanin basina
belirlenir.
Title i belirler.

②

Kesin diktlen
mek kullunulanlar

- 1.) Segment / endis
- 2.) org
- 3.) assume

Veri terimleri

DB, DW, DD, EQU = deyim terimleri

proprinde kullunulan 13 palatlar

tramin lundigi deyim 13'te mis, gibi

deyim lundigi riki. EQU = eleman equator

DUP = kullunulan sonra
parentet i kullunulan deyim
kullunulan ince belirlenir
deyim kullunulan kullunulan
kullunulan kullunulan.

PTR = kullunulan alomina uzerinde
kullunulan kullunulan kullunulan

OR = mov ax, word ptr sagib
mov ah, byte ptr duval

Burak 100 dup (1000000)

Yordam terimleri

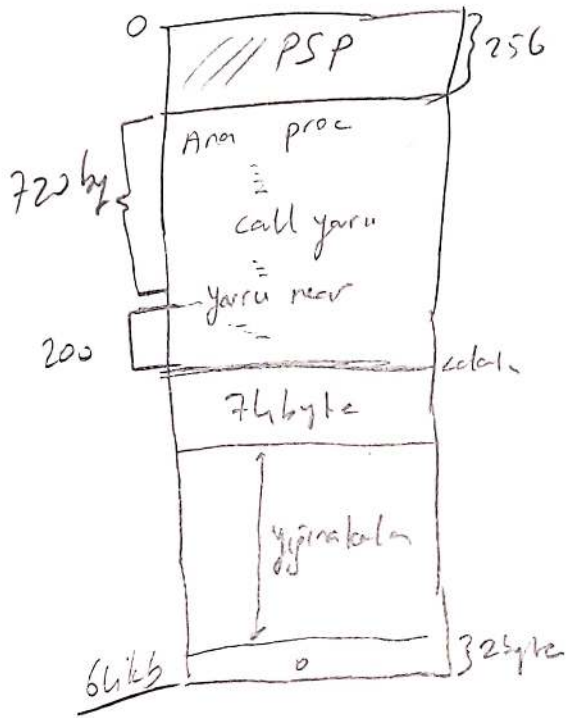
yordam ismi proc {near/far}

ret/retf

yordam ismi endp.

Assembly

S-1 → Com-tipi parlu-iz ana isi-li yralu



$$256 \text{ byte} + 720 \text{ byte} + 200 + 74 + 2 = 1252 \text{ byte}$$

$$65536 - 1252 = 64284 \text{ byte}$$

S2)

1-) Regle parametre alterlabilir (CPSayicib)

2-) Yigin ile alterlabilir parametre

3-) Ortak ver kisi ile param alterlabilir (Cortan/publi)

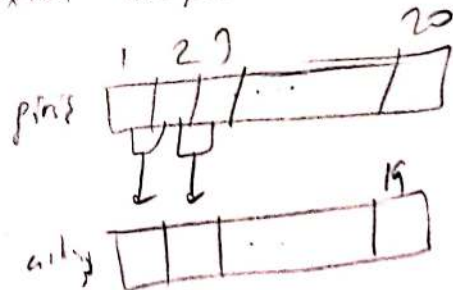
S2) giris isi-li 20 martipinde dicit v. isactisr sayiladur.

klip ede bir 21.1aig obek dicitu bul ve obek dicitu de en buyuk elmanu bul. sonra ax ka dicitu

$$ax = \text{Fobek}(\text{giris}(1), \text{giris}(2))$$

girisin girdiyorak

$$cbyk = \max_{i=1}^{19} (uiku)$$



③

assembly

15.4.18

5-4) Jobeb birliq, rotam yiginda maximal tipinde 2 digir
olir ne katini cap, an qora daw.

Jobeb(a,b)

while (b != 0)

{ t = b

b = a mod t

a = t

}

ret a

public yor atamali

public Jobeb

codes segment proa "retam"
assume cs: codes

Jobeb proc far // Jobeb

Jobeb(a,b)
t ← b
dx: ax / cx

segmentlar logirlovak

bp	bp.sp
dx	ax
cx	bx
bx	bx
ret off	ret
ret segm	ret
size (size)	size
size (size)	size

push bx
push cx
push dx
push bp
mov bp, sp
mov ax, [bp+16]

1: cmp bx, 0

je son

mov cx, bx

xor dx, dx // bolme de katta atlamam wihis dx: ax / cx

div cx

mov bx, dx ; kalan

mov ax, cx

jmp 11

son

pop bp

pop dx

pop cx

pop bx

ret 4

Jobeb endp

codes ends

end

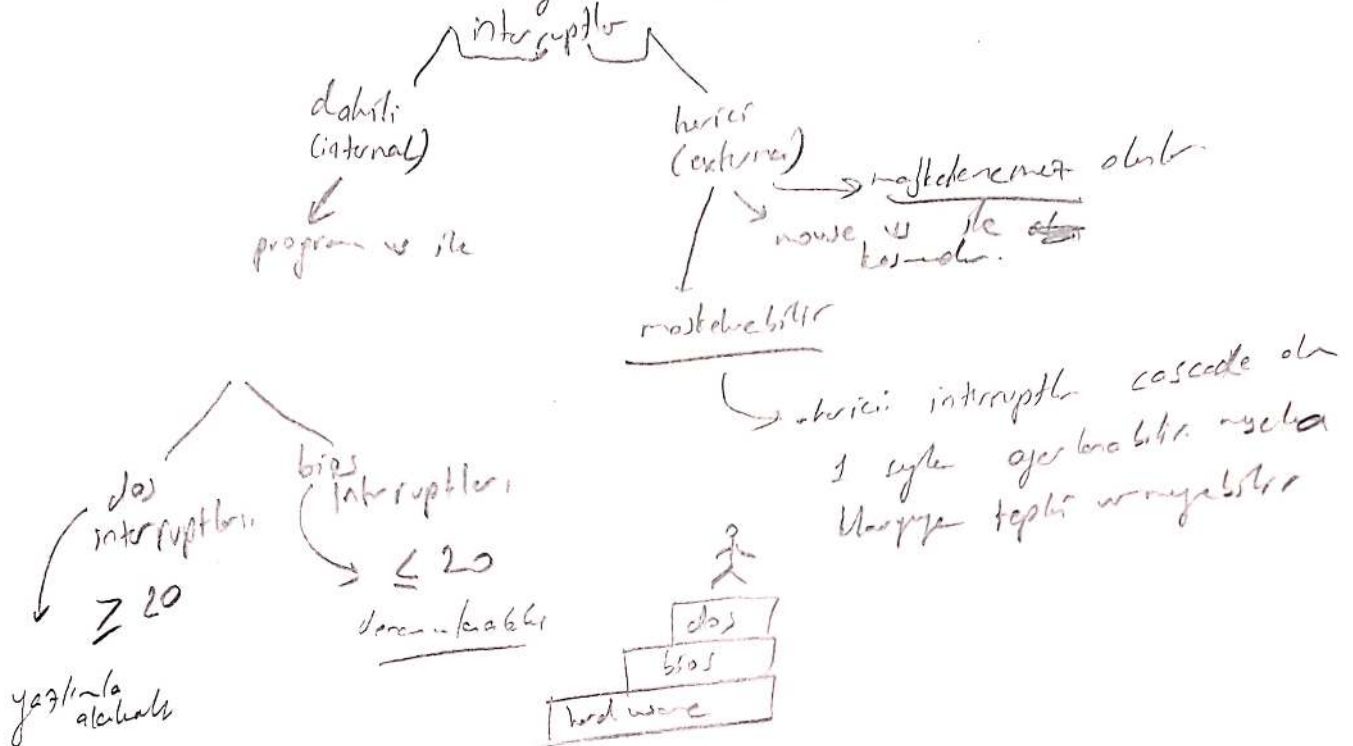
returnu yop sonra

sp = sp + 4 //

bu osongi sd 2'le
pop ax pop yopmak
yovine kullnab, kiritib.
sonacta sp digir
degis 70r.

KBSME MEKANISASI (INTERRUPT)

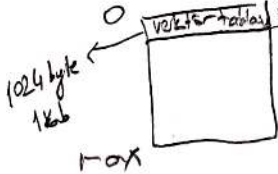
- 1.) Isywaiting 2.) pooling (orubla) 3.) Isy waiting



Vektör Tablosu

→ Bilgiyeğin kriterinin basitçe yer alır

→ Bilgiyeğin kütüphanesi başlıca 3256 adet kesme programının adres bilgisini tutar.
kesim + gövde
kesim.



kesim + göreli
segment + offset
2 + 2 = 4 byte

Durak badan ager naja wok beap dir.

Dataran rendah airnya cek beku dir.
 ④ int 21 h $\rightarrow \underline{21 \times 4}$ yper li-als adresi alir

⑦ întrebarea de domeniul geografic

int 21 k } 1. → dñ → adiclar
yiginde saklanır.
CS int 21 say
IP int 21 of set
bayraklarindaki
bunlar yiginde
saklanır.
push f

5

Assembly

15.4.18

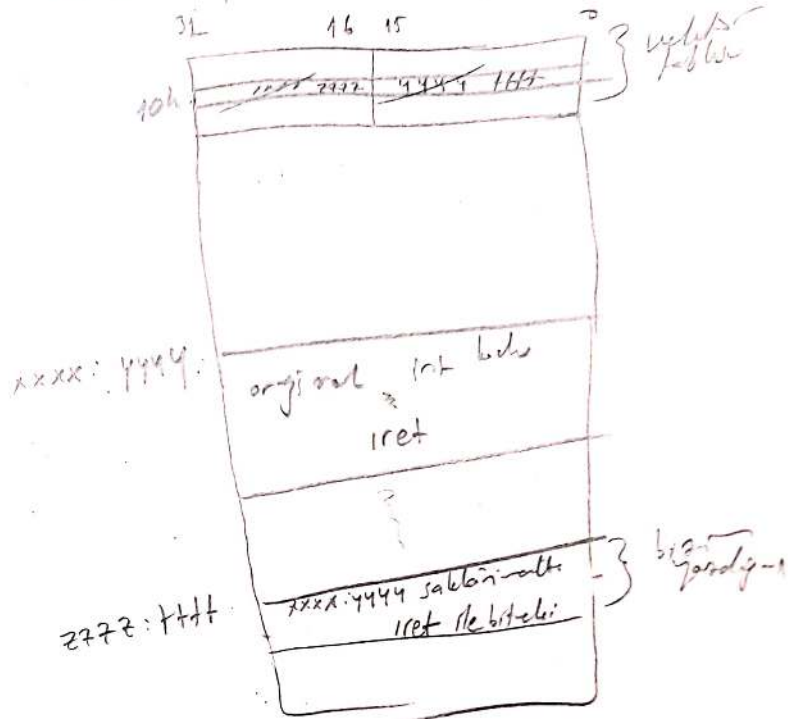
cmp ax, bx
ja 1

int geldi idiyse

cmp buntu bu sayini bitirir.
sonra sonra interrupt yapilir.
bunt yurim karlamaz.

⊗ interrupt registerleri bozabilir
onu koruyan program interrupt
program biter. Stack olarak
bizi stack tutulur. Sere biter.
on o makte stacki bozup korur.

⊗ kasa kasa biter
yasa kisi. crasimada
uyum kiser



tsr kisi yazilmasi

- interrupt yasa kisi?
- sonra interruptin sonu biter
kaybolur?
- interruptin optomunu stajer
olmasi
- yasa kisi makte etelir.
- bios kodlar makte etelir.
- 0's' ye baglamaz sak reset kisi
sonra eski kisi etelir.

①

Assembly

17.4.18

NOT

data segmentte label tanımlandığında bu yer kaplanır!!

sayı dw 10

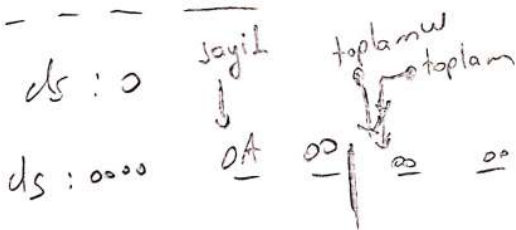
toplama label word

toplama dd 0

70

#TodayIsTuesday

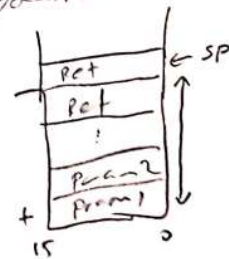
#Assembly



Yordane Parametre Altvirisi

→ en hızlı çalışır.

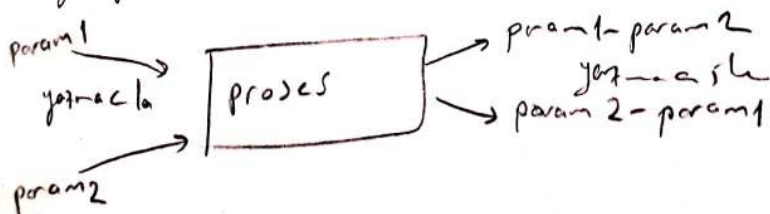
- 1-) Yazma kullanarak (1, 2 adet kullanılarak belli olabilir)
- 2-) Yığın kullanarak (2, 3, 4 adet kullanılarak yığın işte olabilir)
- 3-) Ortak veri alanı/diğerleri kullanarak (Nadiren parametre alınabilir)



⊕ Yığınla ilgili Block BP kullanılır

1-) Yazma ile parametre altvirisi

mesela 2 byte büyüklüğünde diğer yordane parametre olarak kullanılabilir. Bu yordan da yazma ile 2 değer dışlanabilir



②

Assembly

17.4.18

A proc	B proc
Ax ← param1	Ax } selin parametrel
Bx ← param2	Bx }
	???
call L	Ax ← (Ax - Bx) } dönen parametrel
ax ← param1 - param2	Bx ← (Bx - Ax) }
bx ← param2 - param1	

?? = sub ax, bx
neg ax
mov bx, ax
neg ax

Ⓢ Bunu programla, myproc = 17.4.18a

my ss segment para stack 'yigin'

dw 12 dup(?)

my ss ends

my ds segment para 'dest'

param1 dw 5

param2 dw 10

p1-2 dw ?

p2-1 dw ?

my ds ends

Sf247

solu
mov ax, [bp+8]
mov [bp+6], ax } solu

yazac in yazac out

yigin in yigin out

yigin in yazac out

55

35

①

Assembly

24.6.18

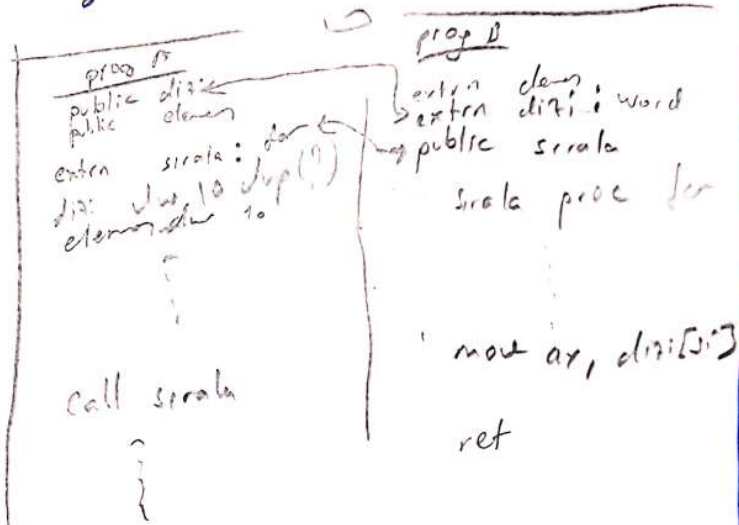
```

push cx
push ax
push bx
push bp
mov , bp, sp
mov ax, [bp+14]
mov bx, [bp+12]
mov cx, ax
sub ax, bx
sub bx, cx
mov [bp+10], ax
mov [bp+12], bx
pop bp
pop bx
pop ax
pop cx

```

BP	← SP, BP
BX	+2
AX	+4
CX	+6
ret offset	+8
ret segment	+10
param 2	+12
param 1	+14

! Lucretia brun !!



*Kesim formlerinden
istte yapılıyor extern public:byts

⇒ Byol ver repitube, stackle
ve ayrı değişkenle ⇒ nerede
dizi kaç elemanlı bu ver

⇒ hocanın kodları hucak
bitirildi

CODE1er 24.06.18

page = 256, 30
for bitirildi

⇒ MAP dosyasına bak pls.

Kod kesim-i ni birleştirme için
ayrı Bİ-le yapıyoruz herşeyi
⇒ data kesim-i adı farklıymiş
olmalı.

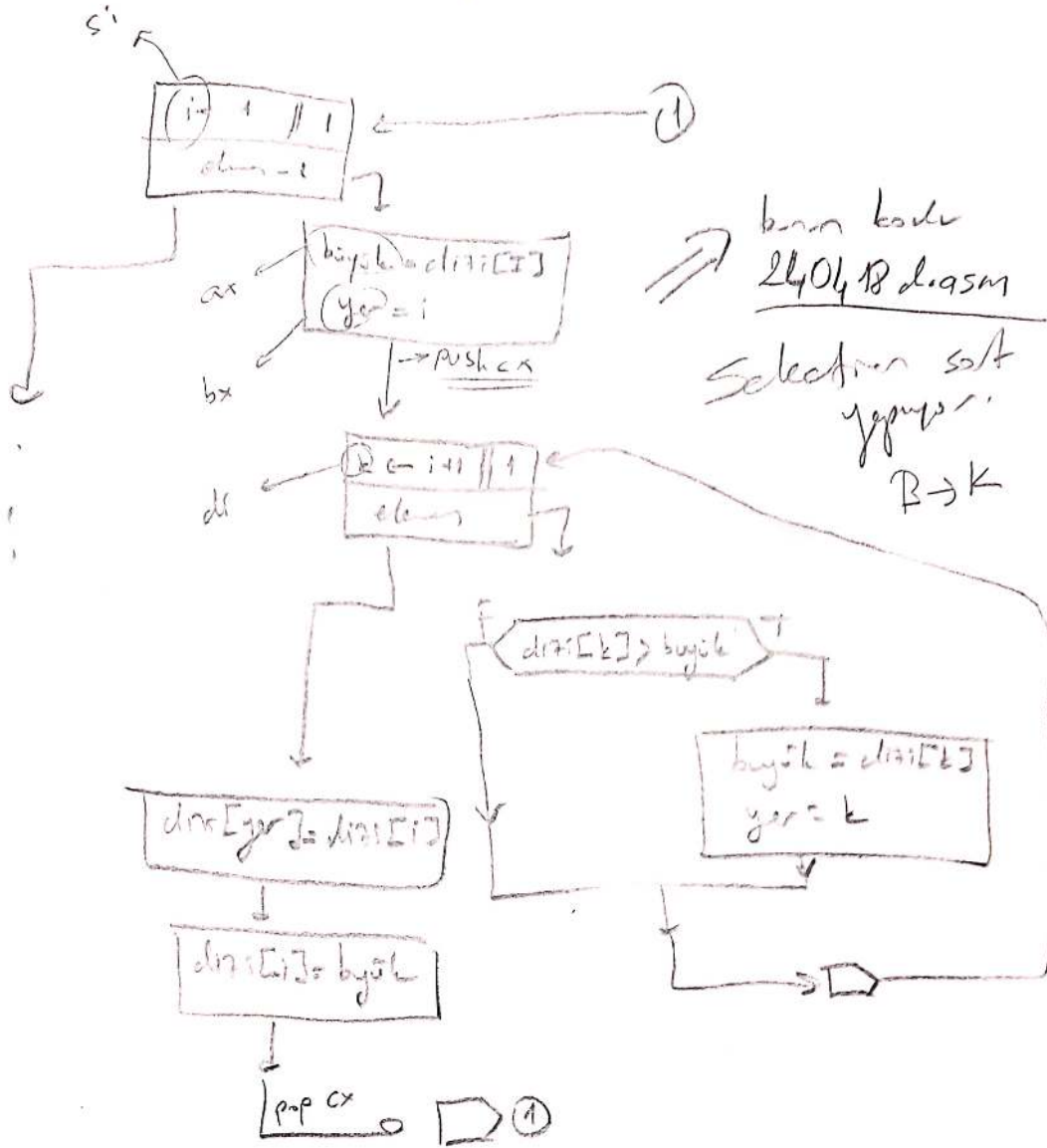
⇒ Kod segmente işi bak pls.

⇒ co- tınısı değişti exe makul.

②

Assembly

24.4.18



bu kodu
24.04.18 d. asm
Selection sort
yapıyor.
B → K

parametre girilen
register
yığın (N tane word)
yığın
ortak veri
ortak veri
or

gelen
register
yığın (M tane word) $M \leq N$
yığın
ortak veri
register

③

Assembly

24.4.18

Macro =

< > ⇒ yolları realde.
opsiyoneldir
olmayabilir bululabilir.

makrolsmi MACRO

<param1, param2...>

ya? macro msg 1

lokal t1, t2
{
}
memoria

endm ⇒ sende return yok!!

endm
*içeride dâim
felen olabılır, loop
jump dâim felen...
labeller vs.

local yapmak makro
sınırlar labellerden

başka labellerle karışmasın
engeller.

yardım göre kez
çapırlması başka
başlang. makro
sırası değil başka
başlang.

yardım zaman kaybı
yapmak makro
dâim atomos nâ
yapılandırıldığında
zaman kaybı yapmaz

macro sıralıdır olmalı
farklılar malı. ki
farklıdır.

macroyu dâim sıralı
!!!

makrolsmi {
makro kod bloğu
kopçaları, kod
hiçbir kriterle sınırlı

makrolsmi {
makro kopçaları
}

makrolsmi {
}

(4)

Assembly

26.6.18

26.6.18.01 de herici Macrover.

26.6.18.01 de espedi-6 colst.

include 26.6.18.01 yazılı.

lea dx, txt1

mov ah, 09h

int 21h

① parametre alması register
yazılı 2 dijital olarak

Bu dijitalın data sign the
temli olarak.

② macro yuclen folklor

③ buldigena buldigena psikoz

1400e cutim 15.000
kuruk dalın 15.000
akır, dıgıyın, 15.000



11

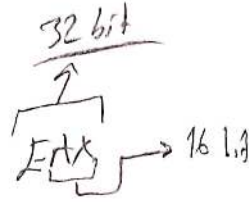
Assembly

22.5.18

extern fonk protolipi

```
int main()
{
    ...
}
```

bulun
sf'le
olacagini
bilgiyecek



```
return 0;
}
```

stacke atilan sirasi

Toplama (dizi, n);



* yatin yordam
cagirlilar.

"C"
de ← sirali
iletiliyor.

sonu sirali
BP ile bulacak.

section .data

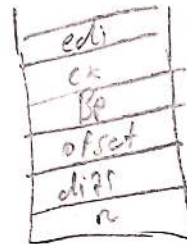
public

section .text
global Toplama.

Toplama:

```
push ebp
push ecx
push edi
mov ebp, esp
mov edi, [ebp+16]
mov cx, [ebp+20]
```

```
l1: → xor eax, ecx
      add eax, [edi] → ax = ax + dizi[i]
      add edi, 4
      loop l1
      pop edi
      pop ecx
      pop ebp
      ret
```



sonu muhakkak

ax reg'i uzerinden
dorecek.

returnden kimen onca

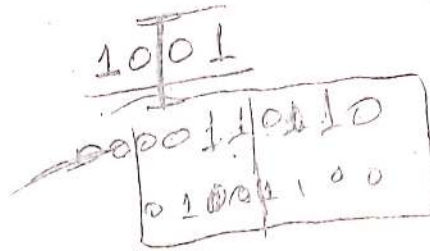
gonu ax'te olmal.

Assembly

Stackle var. hysgi mishi.

- stack's source offset utilizes write near protection.

FOR A



101 $\Rightarrow$ 1011 result? sayk

1101 — 5hr 12

~~aynalamo~~ aynalamo B lami yegitlik.

[illegible]

shr
rel

assembly ve ç'in brake tellerini evde çin.
kendin yap.

c koda assembly ghar.

For the house and the house

①

Assembly Self Study 3.6.18

- Makro adının peşine yollar önce tanımlanmalı. Yani ilk o kodlanmalı.
- INCLUDE sözde komutla, ayrı metin içinde yazılıp, programa dahil edilebilir.
- Sablonu;

```
macroismi macro {param1, param2, ...}  
;  
; macro kodları  
;  
endm
```

→ bu kısım opsiyonel.

- Local sözde komutu ile macro içindeki etiketler Unique olur
- Assume sözde komutla kesin esnekliklerde de değiştirilebilir.

ÖR

```
ustAl macro  
local ll  
mov ax, 1  
mov dl, ll  
loop ll  
endm
```

Macro

```
ana proc far  
...  
mov cx, ust  
mov dl, sayi  
ustAl  
mov sonuc, ax  
...  
ret  
ana endp
```

buraya yukardaki kod direkt
kopyala yapıştır yapılır
diğerleri aynı
- kod büyütür,

②

Assembly Self Study 30.18

→ Com tipinde ayrı netin dosyasında macro bulma:

ÖR=

```
include ornek.mac
codesg segment para 'code'
org 100h
assume cs:codesg, ss:codesg, ds:codesg
```

```
! basta proc near
    sil
    yadir mesaj
```

```
ret
basta endp
```

; deyişken tanımlamaları

mesaj db 'Macrolar ornek.mac dosyasında alındı', '\$'

```
codesg ends
```

```
end      basta ; programın başlığı.
```

- ornek.mac dosyası -

İburada birisi macrolar vardır.
; int 10h ekran silme işlemleri.

```
sil macro
    mov cx, 00010h
    mov dx, 184fh
    mov bh, 07h
    mov ax, 0600h
    int 10h
endm
```

; int 21h ekrana yazdır.

```
yadir macro text
    lea dx, text
    mov ah, 09h
    int 21h
endm
```


3

Assembly Self Study 3.6.18

Ana ve Alt Programdaki Kesimlerin Birleştirilmesi

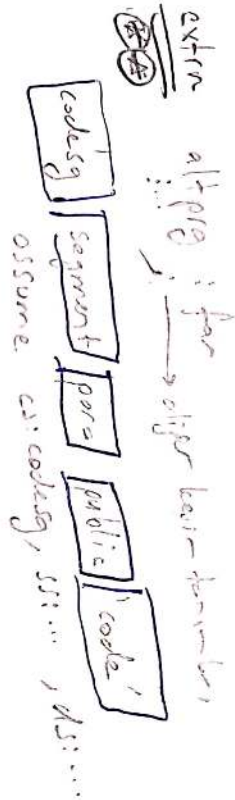
- Birleştirilmez kod kesimi

→ Kesim birleştirilmez, kesimlerin başlangıç adresini belirlemek üzere kullanılır
kaynaklar (alignment) tipinin, kesimlerin gruplandırma için kullanılması
(class) tipinin aynı olması gerekir. AYRICA kesim birleştirilmez
tipinin (combine) public tanımlanması gerekir

ÖRNEK

• Alt program: yordamların public bir şekilde oluyorsa EXTERN sözde-

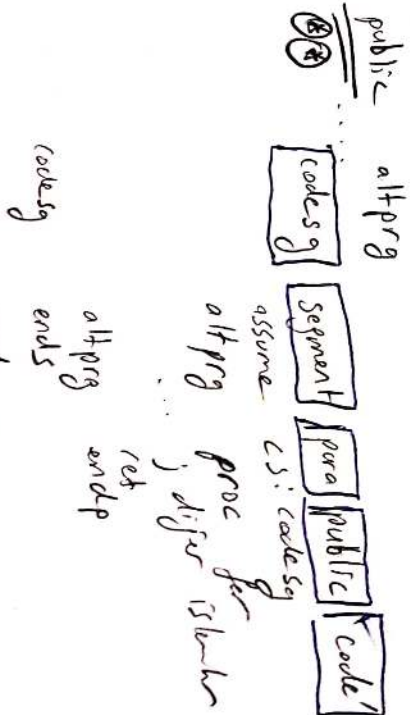
Yordamlar için
iz yapılıyor...



basla proc far
push ds
xor ax, ax
push ax
; diğer işlemler
call altprg
...

ret
basla endp

codeseg ends
end basla ; program başlangıcı noktası



tanımlanmış yordamların göre extrn yordamları: far/near yordamları
→ yordamları: proc far seklinde tanımlanmış ise extrn yordamları: far
→ yordamları: proc near seklinde tanımlanmış ise extrn yordamları: near

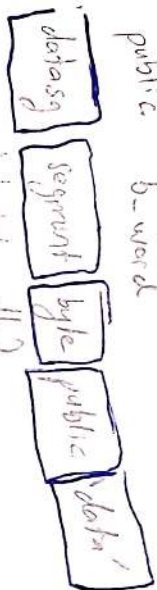
④

Birlestiril-3 Veri Hesiri Kullani

→ full programın raine ta-banıs veri alama(data segment) erişek için
ile extern/public tanımlı kullani.

→ oym, local kullani (code segment) ile old-ın pibi kuralır kurala de aygımlar

ÖRNEK
extern altpirifer, i-word: word
public b-word



i-word db?
i-byte db?
b-word dw 0ABCDH
end s
segment para 'code'
code segment, ds: data segment, ss: ...
disasm

push proc per
push ds
pop error
push ok
... dijir ishar

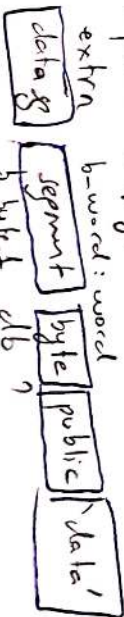
ayniler // etiket ve
attention pls.

call altpir

ret
basta end p

code segment
ends
basta
end basta

public altpir, i-word



data segment
ends
segment para 'code'
assume cs: code, ds: data, ss: ...
altpir proc per
... dijir ishar
ret
altpir end p
code segment
ends

code segment
ends
altpir end p
end

addlabel end altpir deyarli

ayniler kullani aygımlar

⑥

Assembly Self Study

3.6.18

demo

```

push    ds
xor     ax, ax
push    ax
:
ret
basta  endp
codesg  ends
        end    basta

```

----- 0000 -----

```

extra  act:byte, elem:word

stsg   segment para stack 'stack'
dw     10 dup(?)
stsg   ends
dtsg   segment para 'data'
b_byte1 db ?
b_byte2 db ?
dtsg   ends
cdsg   segment para 'code'
assume cs:cdsg, ss:stsg, ds:dtsg

altprg proc far
:       ; döngü ve veri kütlesi için
:       ; gerekli kollar
:
        lea si, ack
:
        ret
altprg  endp
cdsg    ends
        end    altprg

```

⊛ her 2 kod parçası da
basta kolları çalışabiliyor
⊛ ack ya erişilebilirlik için
abj üzerinde çalışırken kollar
dışarı çıkması gerekir.

$1112 \cdot 10^{31}$ → absolute address

- ① Diziyan en boygushe urkischelmini bul.
- ② Verlin sayi oklan, digil mi?
- ③ dizide teher vax yekun vaxda bul.
- ④ Srali dizide vax verimiyokun? bu bul.
- ⑤ Bubble sort ile dizidegi sralayish
- ⑥ Selection sort ile sralayish
- ⑦ Merge algoritmasi
- ⑧ elctra digishlar kullandirilar $a \times b$ sadecce odur subit.
- ⑨ $a+b=1000$ $a \geq b$ olar abe syis.
- ⑩ $L1[1 \dots N]$, $L2[1 \dots M]$, $L3[1 \dots M \times N]$ srali digishini olatur.
- ⑪ Bir dizi srali mi digil mi?
- ⑫ melumli dizidegi 4 birinchi yerden 2 noqluqunin indishlar?
assembly tutari.

⑬ 64 bitlik 2 sayi

cap

⑭ heap sort ile sralayish

⑮ " " " srala

⑯ prim ile mst

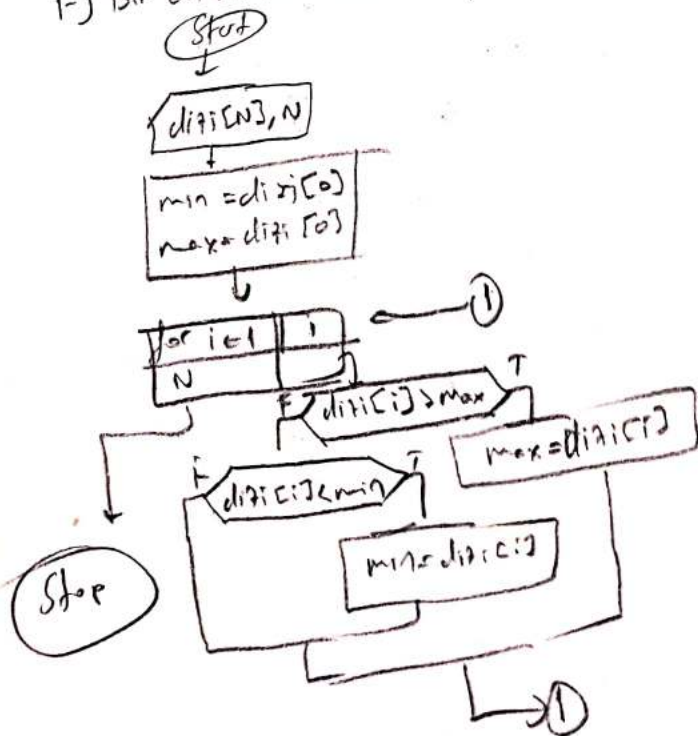
⑰ Quick sort ile srala

⑱ DFS yop

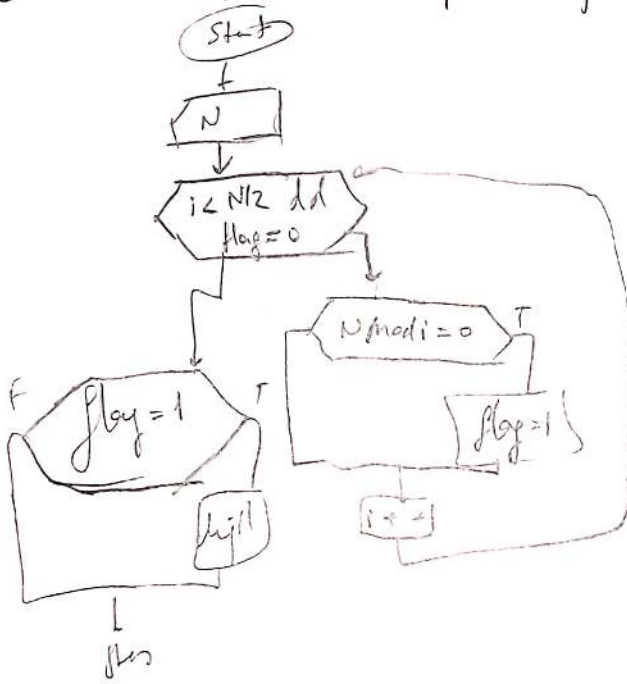
⑲ BFS yop

⑳ Cycle detector yop.

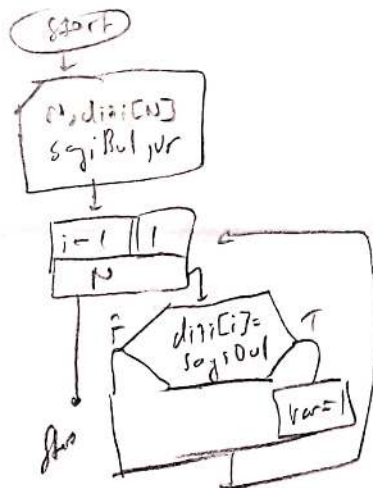
1-) Bir dizinin en boygushe urkischelmini bul



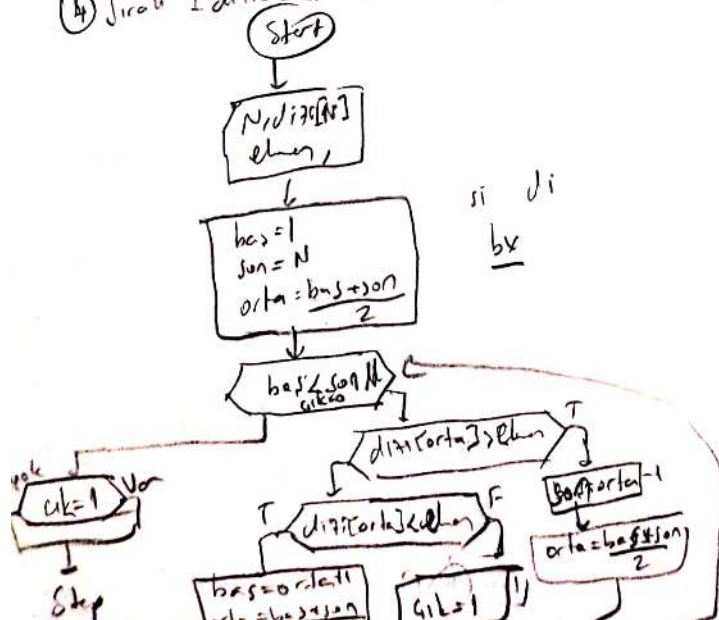
2) Verilen 1 sayının asal olup olmadığını bulunuz



3) Dizi içinde verilen eleman var mı yok mu bul.

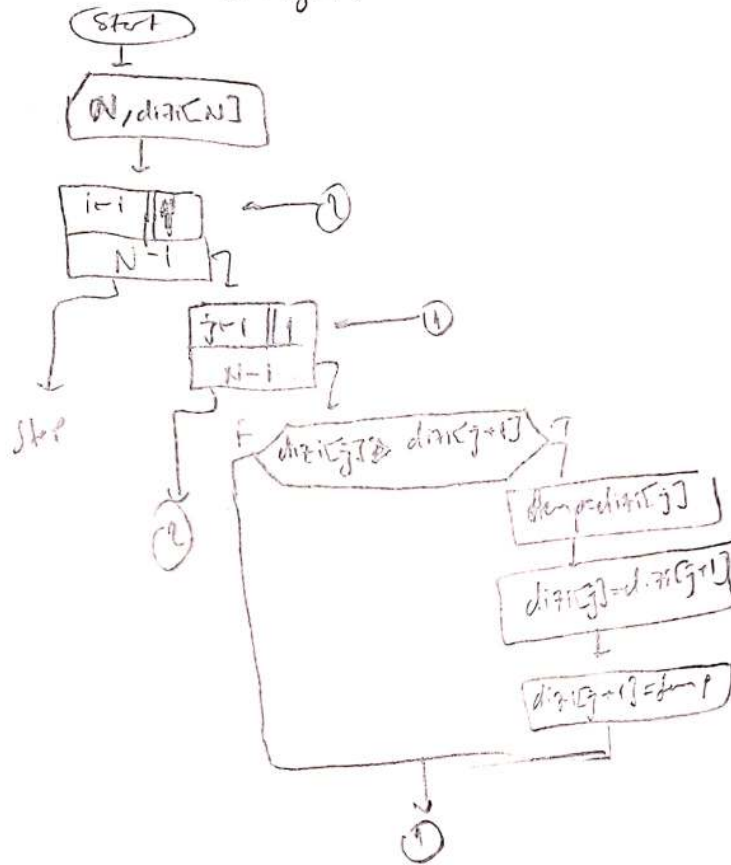


4) Sıralı 1 dizi içinde eleman var mı yok mu bul. (Binary search)

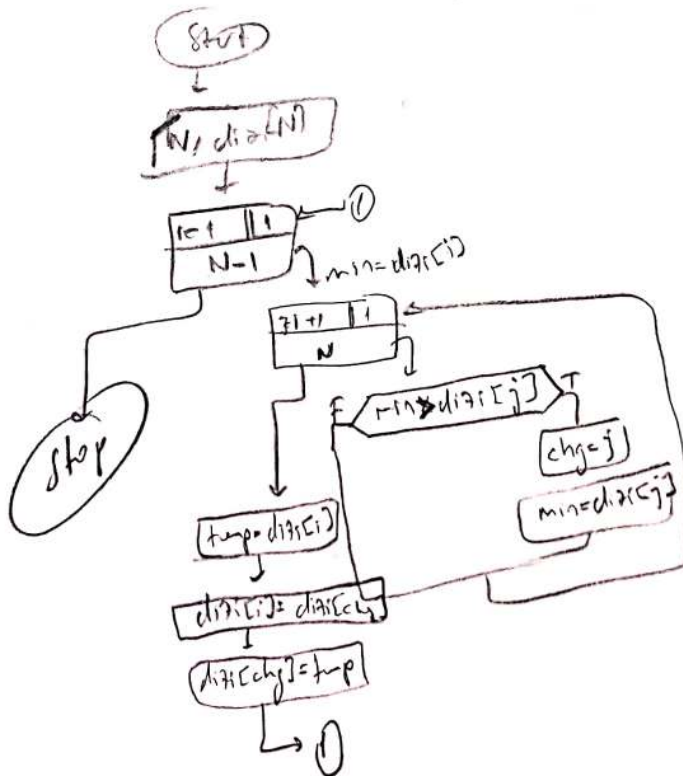


- (3) -

⑤ Bubble sort ile dizi sıralama P.S.

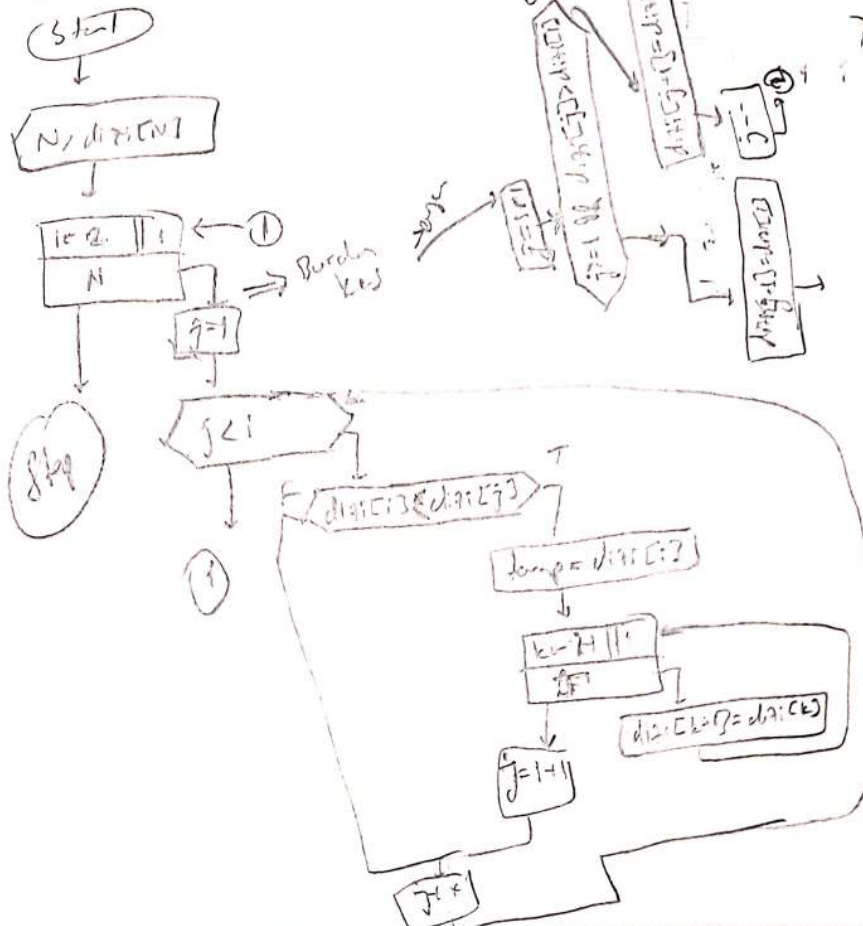


⑥ Kiri: diziye selection sort ile sırala

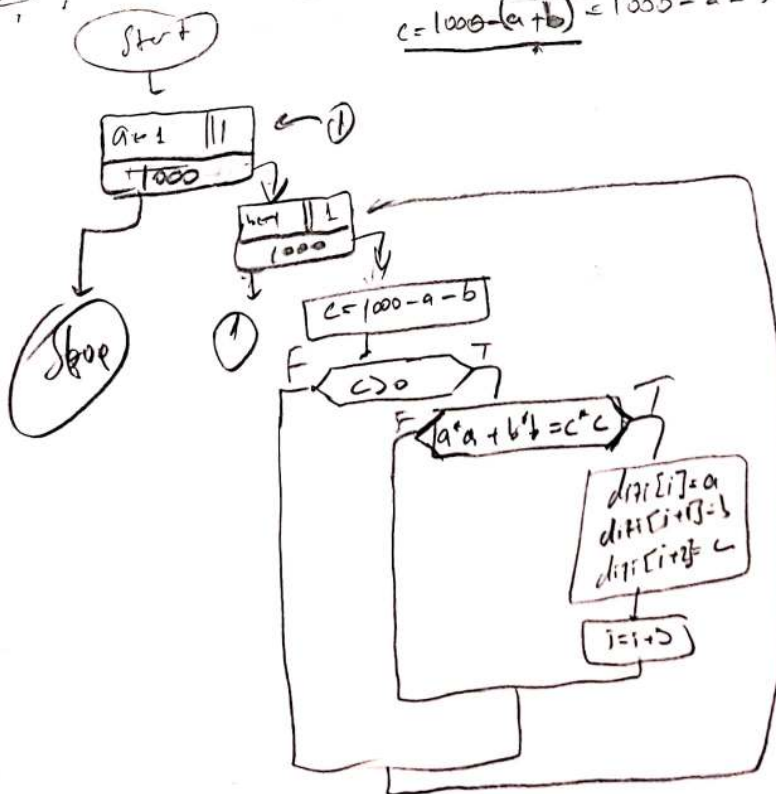


(4)

7) Bir diziyi insertion sort ile sırala



8) $a+b+c=1000$ ver $a^2+b^2=c^2$ olan a, b, c sayılarını bul.
 $c = 1000 - (a+b) = 1000 - a - b$

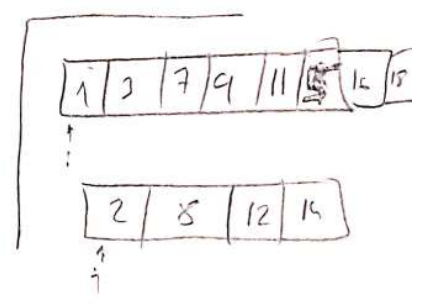
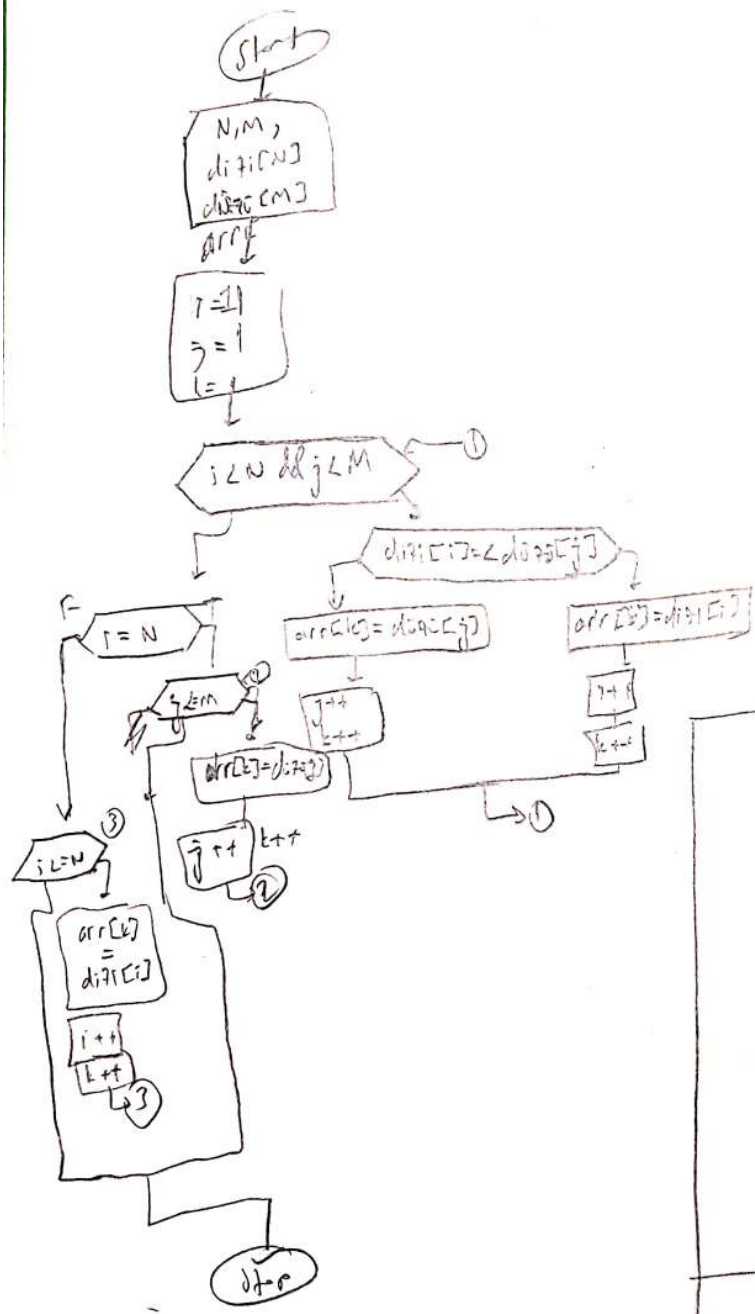


$$999 - a$$

$$1000 - (a + 999 - a)$$

9

$L1 = [1 \dots N]$, $L2 = [1 \dots M]$, $L3 = [1 \dots M+N]$ di fərqli sıralı cluster $L1$ və $L2$ sıralı



10 Bir dist sıralı cluster?

